OPENUSD GEOSPATIAL · CO-SUBMISSION PREVIEW

usdGeospatial

a codeless CRS schema for OpenUSD, proven by two runtimes

schema declares · runtime resolves · one real-world coordinate frame

THE PROPOSAL

A codeless geospatial CRS schema

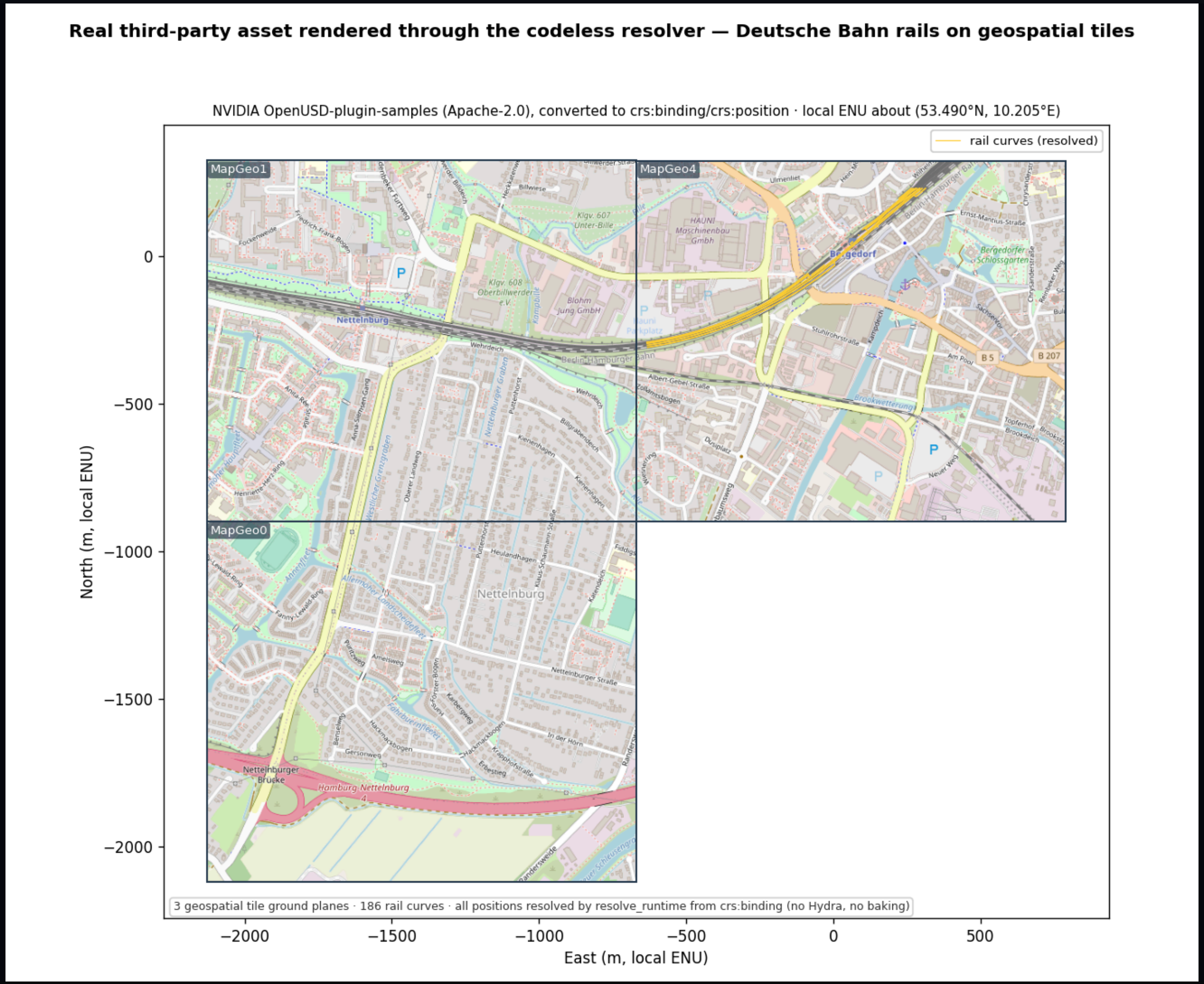
- Pure-data schema: a CRS prim + a binding API, `skipCodeGeneration=true` (no compiled types).
- All resolution behavior lives in the example runtimes, not the schema.
- Same shape as the landed Gaussian / particleField contribution: schema + sample runtime(s) + converter + docs.
- Two runtimes here: Python `resolve_runtime.py` (oracle) and a C++ Hydra scene index that auto-inserts into usdview.

The proofs

Four things an independent reviewer can check.

PROOF · NO OVERFIT

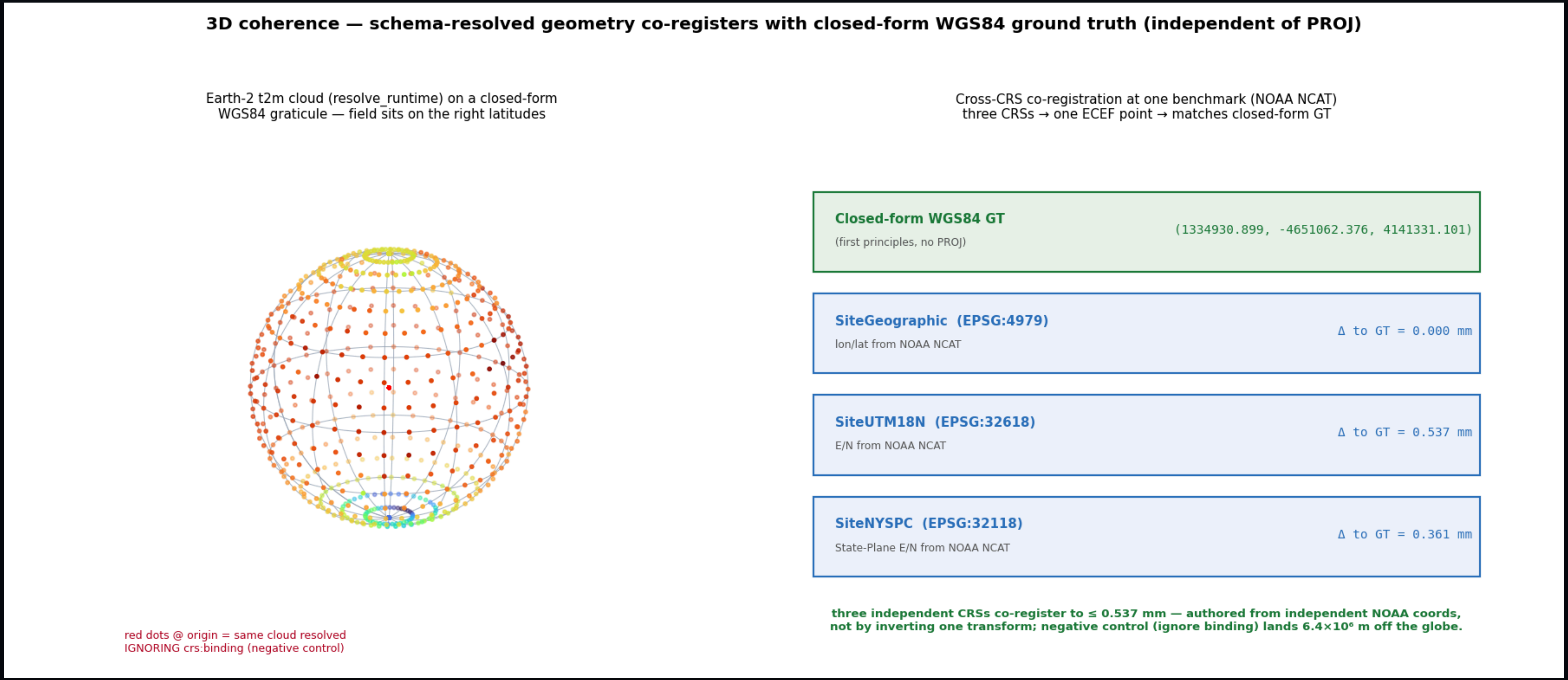
Real third-party asset on real tiles



Matplotlib plot (not a render): an external Deutsche Bahn railway (~1,470 curves) authored in the original Omniverse schema, converted to crs:binding/crs:position, resolved onto three real geospatial tiles near Hamburg.

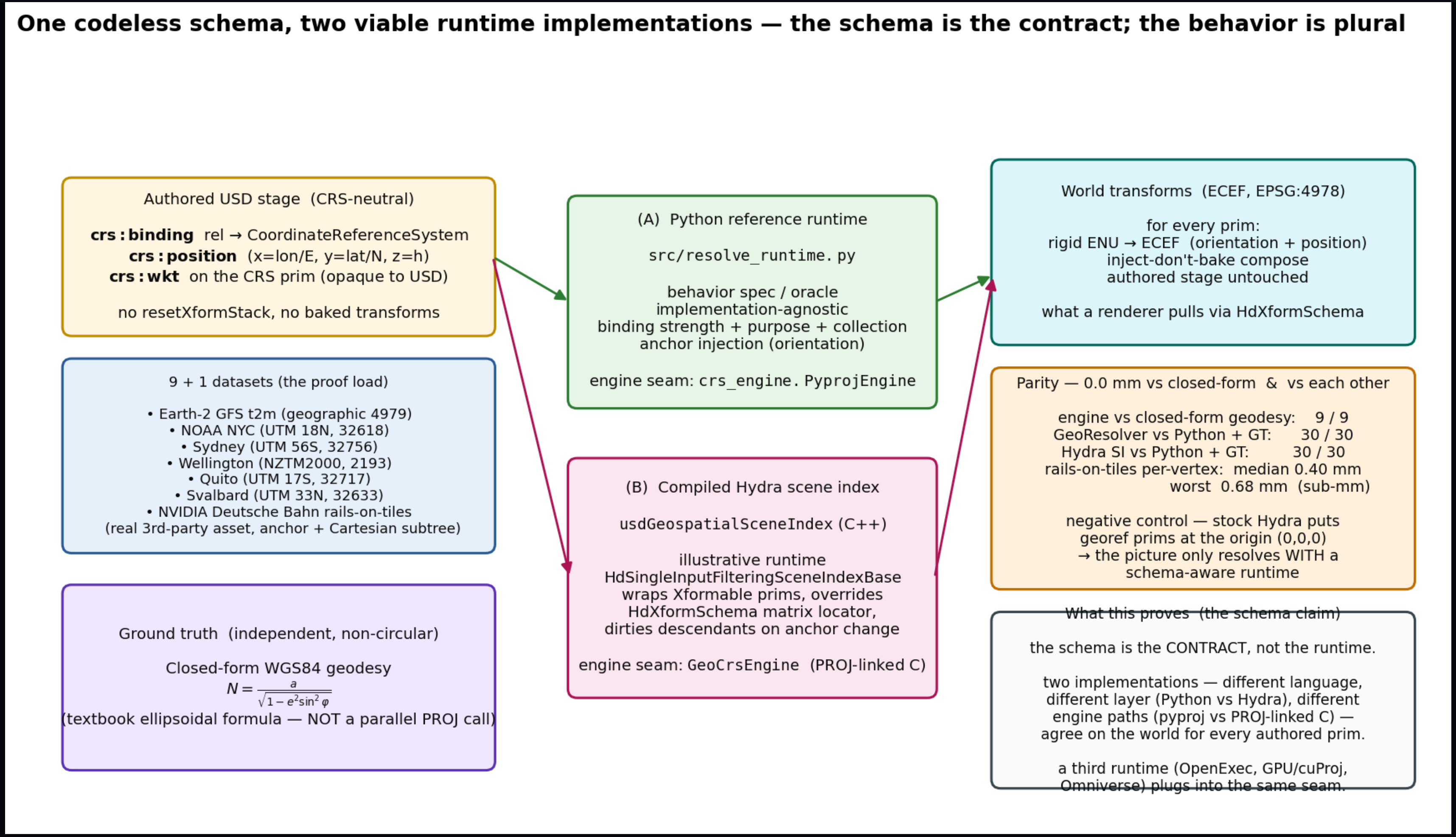
PROOF · CO-REGISTRATION

Three CRSs, one ECEF point



The same monument authored three ways (geographic / UTM 18N / NY State Plane) from independent NOAA NCAT coordinates co-registers to one ECEF point to ≤ ~0.5 mm — vs closed-form WGS84 geodesy, not a parallel PROJ call. Red blob = negative control (bindings ignored).

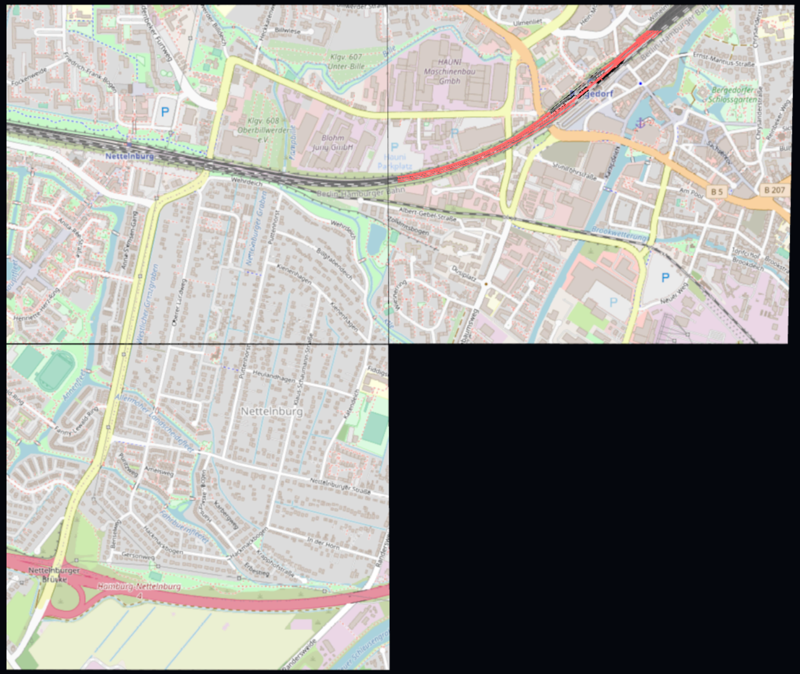
Two independent runtimes, 0.0 mm



The Python reference runtime and the compiled C++ Hydra scene index resolve the same authored stage to the same world, agreeing to 0.0 mm. A third runtime plugs into the same seam.

PROOF · REAL RENDER, AUTO-INSERT

It just works in usdview (Storm)



Real Hydra Storm render (not a plot): with the plugin on the path, the railway auto-resolves at its ECEF position — no app code. Negative control (plugin removed): railway absent. The auto-insert path matches the oracle 30/30 at 0.0 mm (testHydraAutoParity).

The design call

Resolve at runtime; keep the authored scene coordinate-neutral.

REPLACE · WRAP · OR COEXIST?

Resolve, don't bake

- `crs:binding` is a relationship resolved at runtime — not references-as-binding, not a baked `resetXformStack`.
- The authored scene stays coordinate-neutral; a runtime reprojects and composes ancestor transforms.
- This is the question the earlier effort stalled on — replace, wrap, or coexist with the `UsdGeomXformable` stack? The answer here is **coexist**.
- Proven, not argued: the baked Esri scene and our neutral scene land the same corner to 0.0 mm.

COEXISTING WITH USDGEOMXFORMABLE

Anchor injection: inject, don't bake

- A georeferenced anchor with an ordinary Cartesian subtree (a building in local metres) — children have no `crs:position`.
- The runtime injects the anchor's rigid local-frame → ECEF basis (full ENU orientation) at resolve time.
- The subtree then composes under it **as ordinary** **`UsdGeomXformable`** — nothing baked into the layer.
- `resetXformStack` lives only on a transient, computed Hydra data source, never the authored scene.

COMPOSE IN THE CRS-IMPLIED FRAME

Projected vs geographic anchors

- A GEOGRAPHIC/ECEF anchor's child offsets are local metres → compose through the anchor's true-ENU basis (orientation matters; local +Z = ellipsoidal normal).
- A PROJECTED (UTM/State-Plane) anchor's child offsets live in the grid plane → compose IN-PLANE (grid add + reproject), NOT through ENU.
- UTM grid axes differ from true ENU by grid-convergence + point-scale — lifting grid offsets through ENU bends them ~4.86 m over a ~420 m lever.
- Same neutral authored scene; the runtime picks the frame from the bound CRS type. Proven 0.0 mm both ways against closed-form geodesy.

Non-geometric data

A common geospatial case: data with no shape. Visualize it with a composition overlay — without touching or duplicating the data.

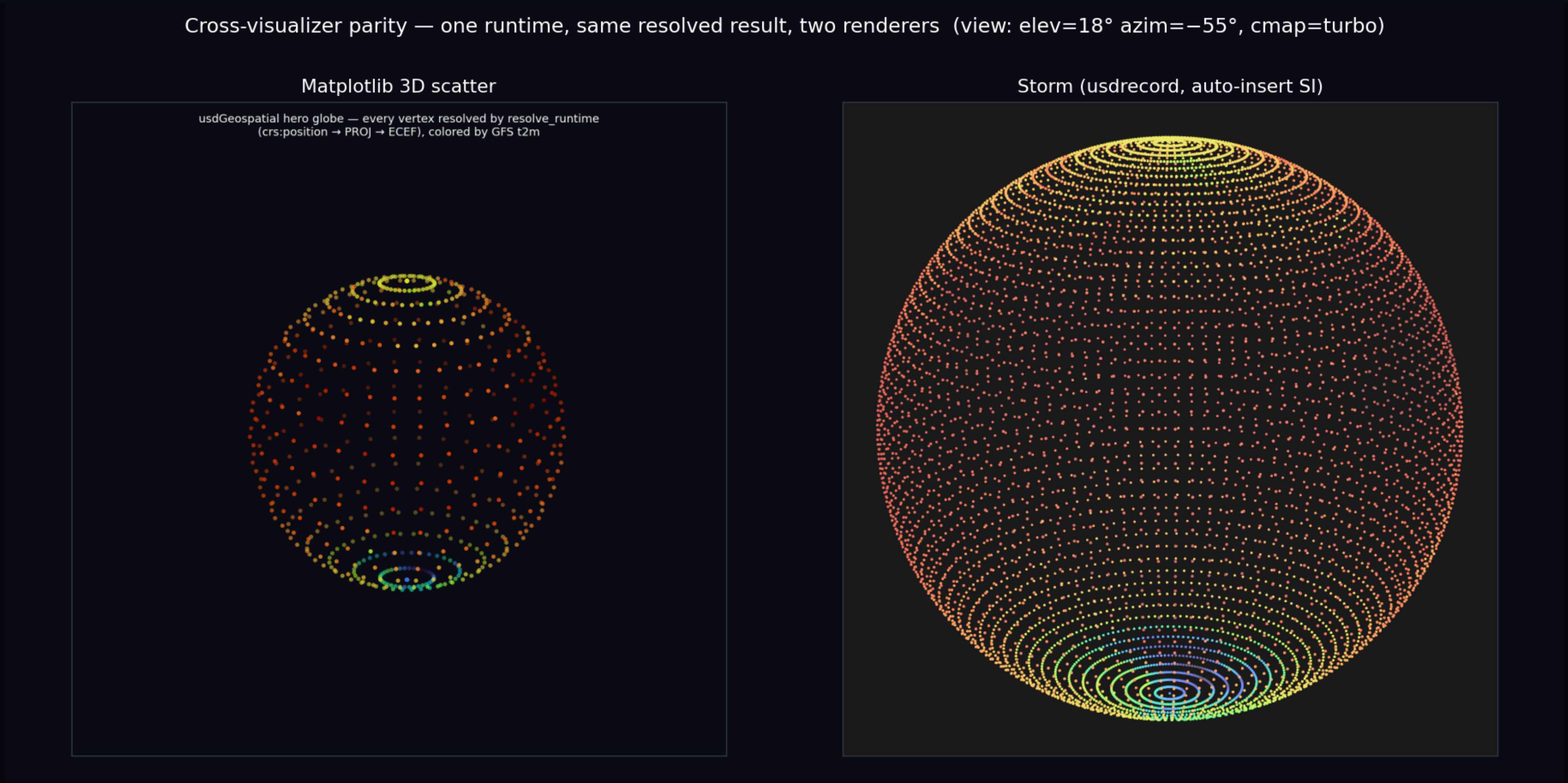
A FIRST-CLASS USE CASE

Non-geometric georeferenced data

- Much geospatial data has NO intrinsic shape: scalar fields (temperature, elevation), sensor/telemetry networks, survey benchmarks. In USD it is a prim carrying `crs:position` + a value, no geometry.
- Visualization is a CHOICE, not part of the data — so add it via USD COMPOSITION: keep the dataset pristine in a base layer; a separate overlay layer subLayers it and adds marker glyphs as geometry-only `over`s.
- ZERO `crs:*` re-authored; the base stays byte-identical; pull the overlay and you are back to pure data.
- The glyphs are Cartesian children at each prim's local origin — the auto-inserted scene index places them via the SAME anchor-injection path as any Cartesian subtree. Marker size is a stated DISPLAY parameter (like a scatter's point size), and the overlay can be sparse.

TWO VISUALIZERS, ONE FIELD

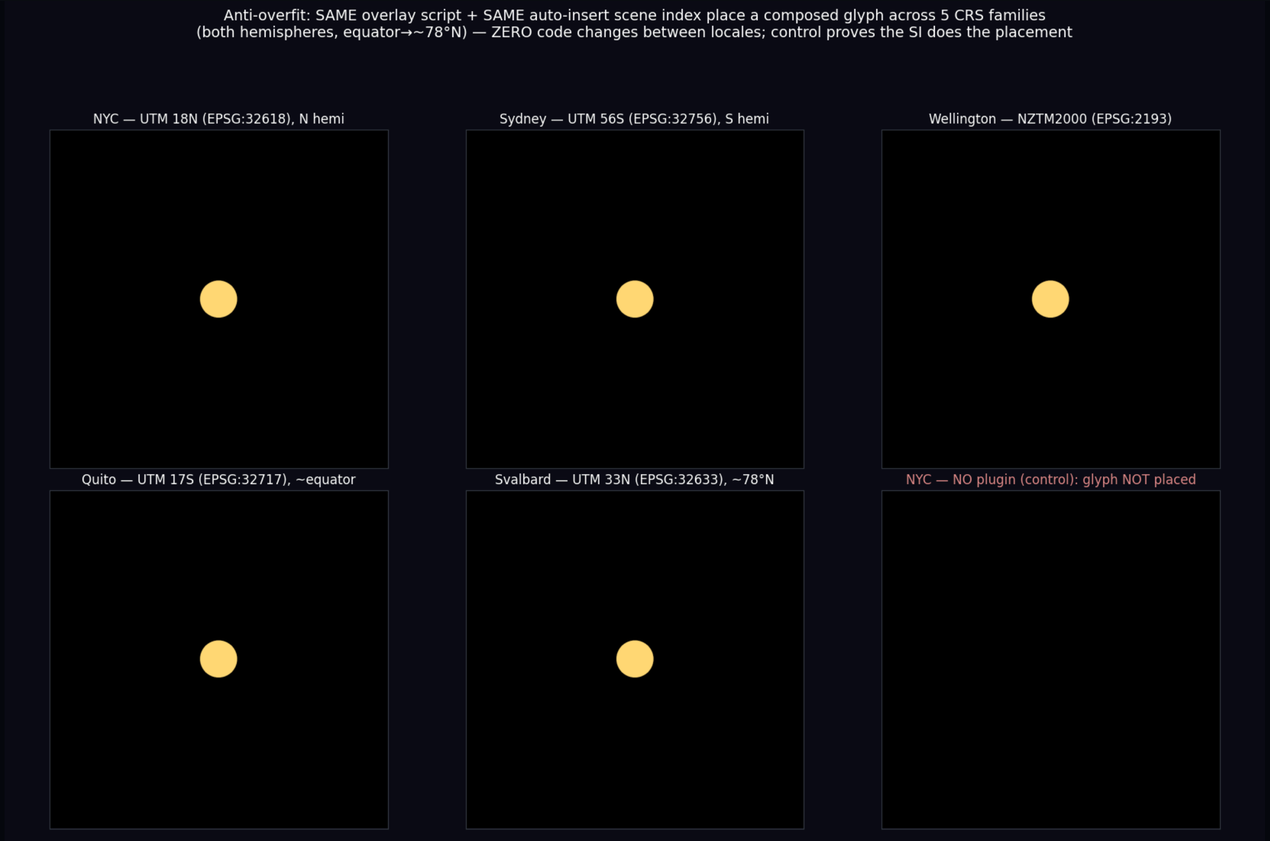
Same non-geometric data, matplotlib vs Storm



The earth2 GFS t2m field (7,320 crs:position samples, no geometry) visualized two ways: a Matplotlib scatter (left) and a Hydra Storm render of a composition-overlay of marker glyphs (right), placed by the auto-inserted scene index. Same resolved positions; glyph style is a disclosed display choice.

PROOF · NOT OVERFIT (RENDERS)

One runtime, five CRS families, both hemispheres



The SAME visualize_field_glyphs overlay + the SAME unchanged scene index place a glyph correctly across NYC (UTM 18N), Sydney (UTM 56S), Wellington (NZTM2000), Quito (~equator), and Svalbard (~78N) — zero code changes between locales. No-plugin controls render empty (the runtime does the placement).

Guard rails

Coexist's costs are asset-structure invariants a validator can enforce.

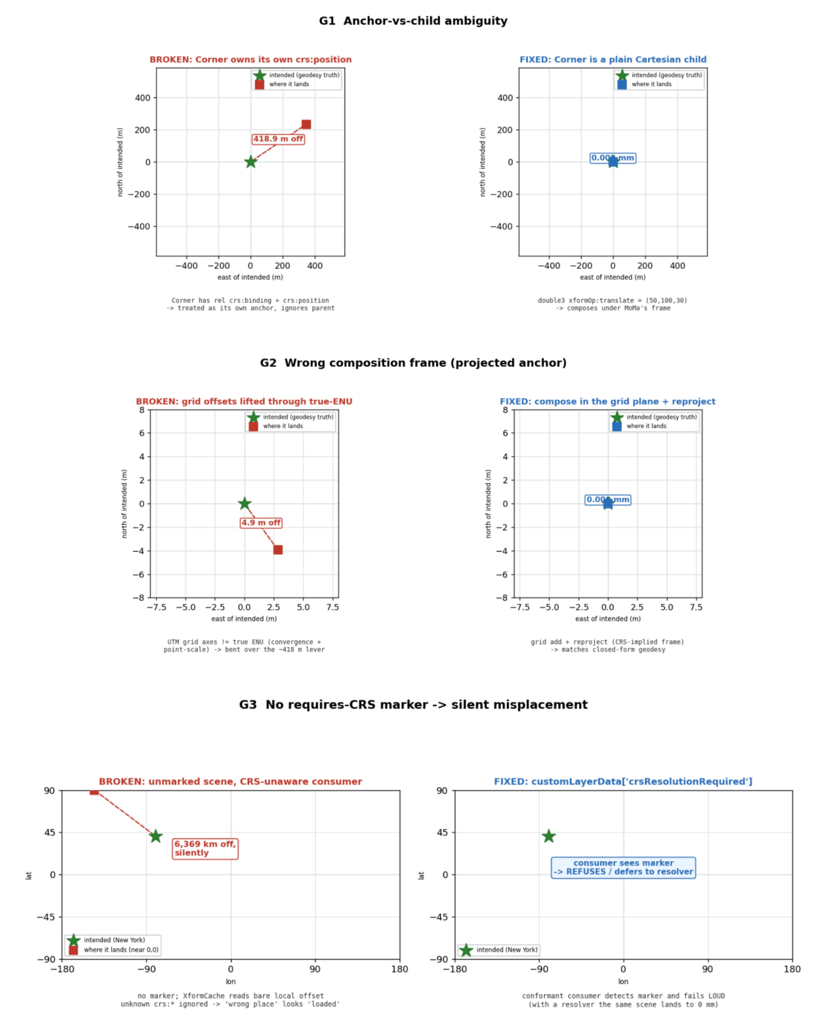
ENFORCEABLE, NOT SHOWSTOPPERS

Guard rails a validator can check

- Coexist has no architectural showstopper — it matches baking to 0 mm when it composes in the CRS-implied frame.
- Its residual costs are a small set of ASSET-STRUCTURE invariants, each mechanically checkable.
- (1) anchor-vs-child is unambiguous; (2) child offsets are authored in the frame the bound CRS implies; (3) a CRS-requiring stage declares it so unaware consumers detect-and-refuse.
- A neutral authored scene PRESERVES the semantic info a validator needs; a baked scene has already collapsed CRS intent into a matrix.

BROKEN → FIXED, MEASURED

What breaks in coexist, and the fix



G1 anchor-vs-child 418.9 m → 0 mm · G2 wrong frame 4.86 m → 0 mm · G3 no marker 6,369 km silent → detect-and-refuse

ANTICIPATED DEBATE

The marker: a tradeoff worth having

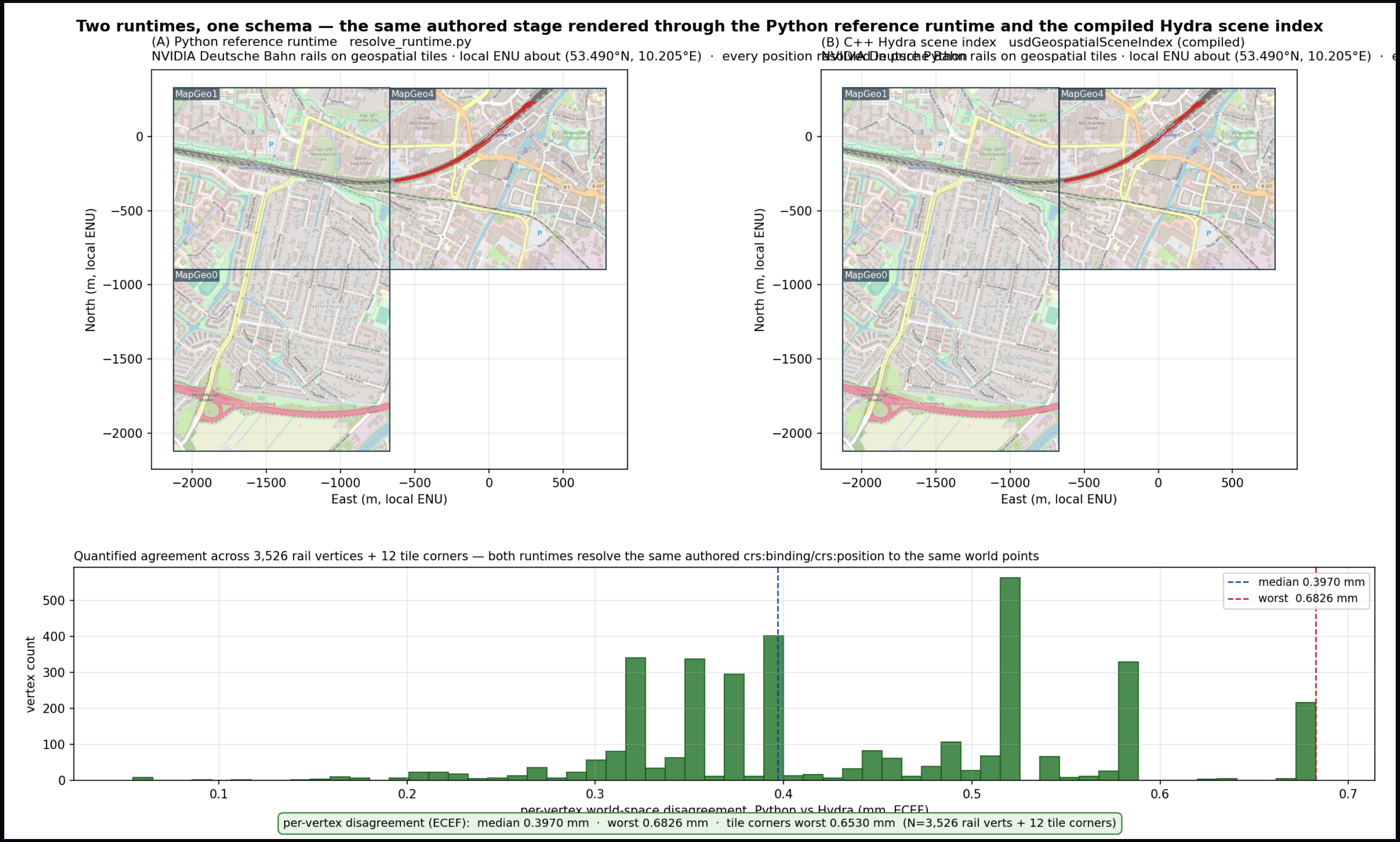
- A 'requires-CRS' marker is the honest cost of coordinate-neutral coexist: it is what lets an unaware consumer fail LOUD instead of silently misplacing.
- Expect pushback — it adds a discovery obligation, and a non-conformant consumer still ignores it (it is a contract, not an enforcement).
- If embraced, the next debate is WHERE it lives: layer metadata (customLayerData) vs a prim-level applied schema.
- Our lean: a stage/layer-level signal for cheap detect-and-refuse, optionally refined per-prim; but this is squarely a WG call.

One schema, two runtimes

The schema is the contract; the behavior is plural.

PROOF · SUB-MM PARITY

Same stage, two runtimes, quantified



Matplotlib plot: Python vs Hydra transforms on the same authored railway stage — 3,526 rail vertices + tile corners agree to median 0.40 mm, worst 0.68 mm. fig_runtime_parity.py fails the build if disagreement exceeds 1 mm.

NO EXACT PRECEDENT — BY DESIGN

Where the Python reference runtime sits

- It's the codeless schema's **executable specification / conformance oracle** — 'given this stage, where does each prim end up?'
- NOT proposed for USD core, NOT a runtime dependency, NOT Python-in-the-render-loop, NOT the prescribed consumer.
- Real consumers (Hydra, OpenExec, Omniverse, GPU cuProj) implement the same contract; the Python is the spec they conform to.
- New part: a runnable reference as the normative behavior for a codeless schema whose behavior is deliberately external.

Open questions

What we'd most like the working group's read on.

FOR THE WORKING GROUP

Open design questions

- 1. Is 'codeless schema + a runnable reference runtime as the behavior contract' the right shape — and where should that reference ultimately live?
- 2. Is **coexist** the right relationship to `UsdGeomXformable` (neutral scene + runtime reconciliation) — versus hooking CRS resolution into `Xformable` directly? Given the head-to-head parity + the guard-rail set, is the residual validation surface acceptable to standardize?

What we need from you

Concrete asks so the next iteration is grounded in your workflows, not our guesses.

ASKS

What we need from you

- 1. The Redlands BIM prototype scene + its validation script, so we can add a second real, contributor-authored case beside the multi-CRS POC.
- 2. Confirmation / correction of the driving workflows: which of AECO site placement, multi-source GIS twins, multi-zone infrastructure, and geodetic sim must 'coexist' survive first?
- 3. A read on the guard-rail set as a conformance surface (anchor-vs-child, compose-in-CRS-frame, a 'requires-CRS' stage marker + detect-and-refuse) and appetite for a codeless validator plugin.
- 4. Where the reference runtime should live, and whether to lift the 'resetXformStack' *semantic* out of the authored-layer text into a documented behavior contract multiple runtimes honor.